



GENERALITAT  
VALENCIANA

Conselleria d'Educació,  
Universitats i Ocupació

ceedcv CENTRE ESPECÍFIC  
D'EDUCACIÓ A DISTÀNCIA DE  
LA COMUNITAT VALENCIANA

# Unidad 01 — Python básico

## Bloque 03

2º CFGS DAM · Programación de Servicios y Procesos

Sergi García Barea



## La idea en una frase

“

El **control de flujo** decide qué código se ejecuta y cuándo: con `if` el programa toma decisiones, con `for` y `while` repite tareas y con `try/except` sobrevive a los errores.



## Condicional: `if / elif / else`

```
una_variable = 5

if una_variable > 10:
    print("Es mayor que 10")
elif una_variable < 10:    # elif es opcional
    print("Es menor que 10")
else:                    # else también es opcional
    print("Es exactamente 10")
```

### Reglas clave

| Regla                   | Detalle                                                                                       |
|-------------------------|-----------------------------------------------------------------------------------------------|
| Indentación obligatoria | La sangría (4 espacios) define los bloques de código.                                         |
| Encadenamiento          | Tantos <code>elif</code> como necesites; solo se ejecuta el primer bloque <code>True</code> . |



La indentación es obligatoria: Python usa la sangría para saber qué código pertenece a cada bloque.



# Condicionales encadenados

Ejemplo práctico de evaluación de notas según rangos:

```
nota = 7

if nota ≥ 9:
    print("Sobresaliente")
elif nota ≥ 7:
    print("Notable")
elif nota ≥ 5:
    print("Aprobado")
else:
    print("Suspenso")

# Salida esperada: "Notable"
```

| Cláusula | Función                                                                                      |
|----------|----------------------------------------------------------------------------------------------|
| if       | Evalúa la primera condición; si es <code>True</code> , ejecuta su bloque y salta el resto.   |
| elif     | Alternativa ( <i>else if</i> ); solo se evalúa si las anteriores fueron <code>False</code> . |
| else     | Caso por defecto, cuando ninguna condición se cumple.                                        |



Solo se ejecuta el primer bloque cuya condición sea `True`.



# Bucle for y la función range()

El bucle `for` itera sobre cualquier **iterable**: listas, tuplas, strings, diccionarios, rangos...

```
for animal in ["perro", "gato", "ratón"]:
    print(f"{animal} es un mamífero")

for i in range(4):
    print(i) # 0, 1, 2, 3
```

## Fórmulas de range(inicio, final, paso)

| Sintaxis                               | Ejemplo                      | Secuencia     |
|----------------------------------------|------------------------------|---------------|
| <code>range(n)</code>                  | <code>range(4)</code>        | 0, 1, 2, 3    |
| <code>range(ini, fin)</code>           | <code>range(2, 6)</code>     | 2, 3, 4, 5    |
| <code>range(ini, fin, paso)</code>     | <code>range(0, 10, 2)</code> | 0, 2, 4, 6, 8 |
| <code>range(ini, fin, paso_neg)</code> | <code>range(5, 0, -1)</code> | 5, 4, 3, 2, 1 |

1 2  
3 4

# Iterar con índice usando `enumerate()`

Para acceder a la vez al índice y al valor, usa `enumerate()` :

```
frutas = ["manzana", "plátano", "cereza"]
```

```
for i, fruta in enumerate(frutas):  
    print(f"{i}: {fruta}")
```

```
0: manzana  
1: plátano  
2: cereza
```

Sin `enumerate()` (tradicional)

```
for i in range(len(lista)):  
  
    print(i, lista[i])
```

Con `enumerate()` (pythonico)

```
for i, val in enumerate(lista):  
  
    print(i, val)
```



**Prefiere `enumerate()` :** evita contadores manuales y expresa mejor la intención.



# Bucle `while` y control de ejecución

El bucle `while` itera mientras la condición sea `True`:

```
x = 0
while x < 4:
    print(x)
    x += 1 # Incremento (equivalente a x = x + 1)

# Salida: 0, 1, 2, 3
```

## ⚠ Cuidado con los bucles infinitos

Si la condición nunca llega a `False`, el programa no acaba nunca:

```
# ¡Bucle infinito!
while True:
    print("Esto no para nunca...")
```



La condición debe poder llegar a ser `False`. Si bloqueas la terminal, interrumpe con `Ctrl + C`.



# Manejo de errores: try / except / finally

Python lanza excepciones cuando algo falla. Puedes capturarlas para que el programa no se rompa:

## Captura básica, con variable y múltiple

```
try:
    resultado = 10 / 0
except ZeroDivisionError:
    print("No se puede dividir por cero")

try:
    print([1, 2, 3][10])
except IndexError as e:
    print(f"Error: {e}")

try:
    x = int(input("Número: "))
    resultado = 10 / x
except ValueError:
    print("No es un número")
finally:
    print("Siempre se imprime")
```



# Bloques try / except / finally

| Bloque  | Cuándo se ejecuta                       | Uso principal                                   |
|---------|-----------------------------------------|-------------------------------------------------|
| try     | Siempre al inicio.                      | Contiene el código susceptible de fallar.       |
| except  | Solo si se lanza la excepción indicada. | Tratamiento y recuperación del error.           |
| finally | <b>Siempre</b> , haya fallado o no.     | Cerrar archivos, conexiones o liberar recursos. |

```
try:
    f = open("archivo.txt")
    contenido = f.read()
except FileNotFoundError:
    print("Archivo no encontrado")
finally:
    print("Esto se imprime siempre")
```



# Iterables e iteradores por dentro

Un **iterable** es cualquier objeto sobre el que puedes hacer un `for`: listas, tuplas, strings, diccionarios, conjuntos, range...

Un **iterador** mantiene el estado de la iteración y sabe cuál es el siguiente elemento.

```
mi_lista = [1, 2, 3]

# La función iter() crea un iterador a partir del iterable
mi_iterador = iter(mi_lista)

# next() avanza elemento a elemento
print(next(mi_iterador)) # 1
print(next(mi_iterador)) # 2
print(next(mi_iterador)) # 3
# next(mi_iterador)      # Lanza la excepción StopIteration (sin más elementos)
```



Cuando escribes `for x in iterable`, Python crea automáticamente un iterador por debajo usando `iter()` y `next()`.



# Funciones: `def`, `return` y argumentos

Una función es un bloque de código reutilizable que hace una sola cosa. Se define con `def`, se llama con paréntesis y puede recibir argumentos y devolver valores.

```
def sumar(x, y):  
    print(f"x es {x} y y es {y}")  
    return x + y # Sin return, devuelve None implícitamente  
  
resultado1 = sumar(5, 6)      # posicionales (orden estricto) → 11  
resultado2 = sumar(y=6, x=5) # de palabra clave (el orden no importa) → 11
```

| Regla               | Detalle                                                          |
|---------------------|------------------------------------------------------------------|
| <code>def</code>    | Palabra clave + nombre, paréntesis y dos puntos <code>:</code> . |
| Indentación         | El cuerpo va indentado a 4 espacios.                             |
| <code>return</code> | Devuelve uno o varios valores (con coma, retorna una tupla).     |



Sin `return`, la función devuelve `None` implícitamente.



# Argumentos variables: `*args` y `**kwargs`

Funciones flexibles que aceptan cualquier cantidad de parámetros:

`*args` — posicionales en tupla · `**kwargs` — nombrados en dict

```
def varargs(*args):  
    return args  
  
varargs(1, 2, 3)    # => (1, 2, 3)  
  
keyword_args(pie="grande", lago="ness")    # => {"pie": "grande", ...}
```

## Combinar ambos y desempaquetar al llamar

```
def todos_los_args(*args, **kwargs):  
    print(f"Aargs (tupla): {args}")  
    print(f"Kwargs (dict): {kwargs}")  
  
todos_los_args(1, 2, a=3, b=4)    # Aargs: (1, 2) · Kwargs: {'a': 3, 'b': 4}  
  
todos_los_args(*(1, 2, 3, 4), **{"a": 3})    # desempaquetando
```



Úsalos solo cuando la función deba aceptar parámetros variables.



# Clausuras (closures)

En Python las funciones son **objetos de primera clase**: se guardan en variables, se pasan como argumentos y se devuelven.

```
def crear_suma(x):  
    def suma(y):  
        return x + y  
    return suma  
  
sumar_10 = crear_suma(10)  
print(sumar_10(3)) # Devuelve 13
```

La función interna `suma` "recuerda" el valor de `x` aunque `crear_suma` ya ha terminado.



Las cláusuras son la base de los decoradores, que veremos en el bloque de POO y módulos.



# Funciones lambda (anónimas)

Funciones anónimas de una sola expresión con sintaxis `lambda args: expresión`:

```
# Lambda declarada y ejecutada "al vuelo"
es_mayor = (lambda x: x > 2)(3) # Devuelve True

cuadrado = lambda x: x ** 2
print(cuadrado(5)) # Devuelve 25
```



Las lambdas son útiles como argumentos de otras funciones. Deben ser cortas y contener una única expresión.



# Funciones de orden superior: `map`, `filter` y

## `reduce`

| Función                       | Qué hace                                                          | Ejemplo                                                                                      |
|-------------------------------|-------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| <code>map(fn, iter)</code>    | Aplica una función a cada elemento.                               | <code>list(map(lambda x: x**2, [1, 2, 3, 4]))</code> → <code>[1, 4, 9, 16]</code>            |
| <code>filter(fn, iter)</code> | Retiene los que cumplen la condición.                             | <code>list(filter(lambda x: x % 2 == 0, [1, 2, 3, 4, 5, 6]))</code> → <code>[2, 4, 6]</code> |
| <code>reduce(fn, iter)</code> | Acumula todo en un solo valor (requiere <code>functools</code> ). | <code>reduce(lambda a, b: a + b, [1, 2, 3, 4])</code> → <code>10</code>                      |

```
from functools import reduce
```

```
numeros = [1, 2, 3, 4]
```

```
suma_total = reduce(lambda a, b: a + b, numeros)
```

```
print(suma_total) # 10
```



`map()` y `filter()` devuelven iteradores perezosos: envuélvelos en `list()`.



# Comprensiones (comprehensions)

Forma concisa y pythonica de generar colecciones a partir de iterables:

## Comprensión de lista

```
# Bucle clásico                # Comprensión (lo mismo en una línea)
cuadrados = []                 cuadrados = [x ** 2 for x in range(5)]
for x in range(5):
    cuadrados.append(x ** 2)

# Con condición integrada (filtro)
pares = [x for x in range(10) if x % 2 == 0] # => [0, 2, 4, 6, 8]
```

## Comprensión de diccionario y de conjunto

```
cuadrados_dict = {k: k ** 2 for k in range(3)} # => {0: 0, 1: 1, 2: 4}
letras = {c for c in "la cadena"} # => {'l', 'a', ' ', 'c', 'd', 'e', 'n'}
```



Las comprehensions sustituyen patrones simples de `for` + `append` con código conciso y pythonico.



# Mini-chequeo — Flujo y funciones

1. ¿Qué imprime `for i in range(3): print(i)` ?
2. ¿Qué pasa si olvidas el `:` al final de un `if` ?
3. ¿Cómo capturas un `ValueError` en un `try/except` ?
4. ¿Cuál es la diferencia entre `*args` y `**kwargs` ?
5. ¿Qué devuelve `map(lambda x: x * 2, [1, 2, 3])` sin envolver en `list()` ?

## Respuestas:

1. Imprime 0, 1, 2 (uno en cada línea). `range(3)` va de 0 a 2.
2. Python lanza un error de sintaxis: `SyntaxError: invalid syntax`.
3. `except ValueError:` o `except ValueError as e:` si quieres guardar el mensaje de error.
4. `*args` recoge argumentos posicionales en una tupla. `**kwargs` recoge argumentos de palabra clave en un diccionario.
5. Un objeto map (un iterable perezoso). Para ver los valores, necesitas `list()`.



## Resumen — Bloque 03

- `if/elif/else` toma decisiones; la indentación define los bloques.
- `for` itera sobre iterables; `while` itera mientras haya condición.
- `try/except` captura errores sin que el programa se rompa.
- Se define con `def`, se llama con paréntesis y se devuelve valor con `return`.
- `*args` y `**kwargs` permiten argumentos variables; las lambda son funciones anónimas de una expresión.
- Las comprensiones crean listas, diccionarios y conjuntos de forma concisa y legible.



# Vocabulario rápido

| Término                                 | Idea general                                 |
|-----------------------------------------|----------------------------------------------|
| Iterable                                | Objeto sobre el que se puede iterar          |
| Iterador                                | Recorre un iterable elemento a elemento      |
| Excepción                               | Error que se lanza durante la ejecución      |
| Indentación                             | Sangría (4 espacios) que define bloques      |
| Comprensión                             | Colección creada con un bucle conciso        |
| <code>def</code> / <code>return</code>  | Definir una función / devolver un valor      |
| <code>args</code> / <code>kwargs</code> | Posicionales variables / nombrados variables |

| Término                 | Idea general                          |
|-------------------------|---------------------------------------|
| Lambda                  | Función anónima de una sola expresión |
| Cláusura                | Función que recuerda su contexto      |
| <code>map</code>        | Aplica una función a cada elemento    |
| <code>filter</code>     | Filtra según una condición            |
| <code>range</code>      | Secuencia de números para iterar      |
| <code>enumerate</code>  | Da (índice, valor) en cada iteración  |
| <code>try/except</code> | Captura errores sin romperse          |

